Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И УПРАВЛЕНИЕ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ, ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОМУ ПРОЕКТУ НА ТЕМУ:

Разработка клиент-серверного приложения для визуализации телеметрии спутника

Студент группы ИУК4-72Б _____ А.А. Боков
(подпись, дата) (И.О. Фамилия)

Руководитель курсового проекта _____ М.И. Былинка
(подпись, дата) (И.О. Фамилия)

Калуга, 2025

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой __**ИУК4**___
_____(_Ю.Е. Гагарин_)
«_05_»____сентября____2024г.

З А Д А Н И Е
на выполнение курсового проекта

по дисциплине ***Компьютерные сети и Интернет-технологии***

Студент группы ***ИУК4-72Б Боков Артём Алексеевич***
(фамилия, имя, отчество)

Тема курсового проекта ***Разработка клиент-серверного приложения для визуализации телеметрии спутника***

Направленность КП ***учебный***
Источник тематики ***кафедра ИУК4***

Задание

Провести анализ требований и технологий разработки программного обеспечения.
Выполнить проектирование программного обеспечения.
Осуществить интеграция компонентов программного обеспечения.

Оформление курсового проекта

Расчетно-пояснительная записка на___48____ листах формата А4.
Перечень графического материала КП (плакаты, схемы, чертежи и т.п.):
− Интерфейс приложения – 1 лист формата А3;
− Блок-схемы основных алгоритмов приложения – 1 лист формата А3;
− Диаграммы – 1 лист формата А3.

Дата выдачи задания «_05_» сентября 2025 г.

Руководитель	05.09.2025	М.И. Былинка
	(подпись, дата)	(И.О. Фамилия)
Студент	05.09.2025	А.А. Боков
	(подпись, дата)	(И.О. Фамилия)

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

КАЛЕНДАРНЫЙ ПЛАН
на выполнение курсового проекта

по дисциплине ***Компьютерные сети и Интернет-технологии***

Студент группы ***ИУК4-72Б Боков Артём Алексеевич***
(фамилия, имя, отчество)

Тема курсового проекта ***Разработка клиент-серверного приложения для визуализации телеметрии спутника***

№	Наименование этапов	Сроки выполнения этапов		Отметка о выполнении	
		план	факт	Руководитель	Куратор
1	Задание на выполнение	1-я нед.			
2	Выполнение логического проектирования программного обеспечения	10-я нед.			
3	Выполнение и окончательное оформление графической части и расчетно-пояснительной записки	14-я нед.			
4	Защита	17-я нед.			

Студент _____ 05.09.2025г. Руководитель _____ 05.09.2025г.
(подпись, дата) (подпись, дата)

РЕФЕРАТ

Расчетно-пояснительная записка 48 с., 12 рисунков, 2 таблицы, 16 источников.

Разработка клиент-серверного приложения для визуализации телеметрии спутника.

Объектом разработки является клиент-серверное программное обеспечение, предназначенное для приёма, передачи, хранения и обработки телеметрических данных, поступающих от внешнего источника.

Целью курсовой работы является разработка клиент-серверного приложения, обеспечивающего автоматизированный сбор, обработку и отображение телеметрической информации, передаваемой по протоколу TCP, с последующим хранением данных в оперативной памяти и предоставлением их пользователю в удобной для анализа форме.

Поставленные в рамках курсовой работы задачи решаются путём проектирования и реализации клиент-серверного взаимодействия между источником телеметрических данных, серверным приложением и клиентской частью с использованием современных средств и технологий программирования, а также выбранной архитектуры программного обеспечения

СОДЕРЖАНИЕ

РЕФЕРАТ 3

СОДЕРЖАНИЕ 5

ВВЕДЕНИЕ 5

1. АНАЛИЗ ТРЕБОВАНИЙ И ТЕХНОЛОГИЙ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ 7

1.1. Основные требования к разрабатываемой системе 7

1.2. Анализ аналогов и прототипов 8

1.3. Обоснование выбора инструментов и платформы для разработки клиентской части 11

1.4. Обоснование выбора инструментов и платформы для разработки серверной части 12

2. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ 16

2.1. Разработка структуры системы 16

2.2. Структура телеметрического кадра 19

2.3. Визуализация структуры проектов 23

2.4. Блок-схемы основных алгоритмов проекта 26

3. КОНТРОЛЬ КАЧЕСТВА И ИНТЕГРАЦИЯ КОМПОНЕНТОВ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ 35

3.1. Интерфейс клиентской части 35

3.2. Руководство пользователя 41

3.3. Руководство администратора 41

3.4. Руководство администратора 42

ЗАКЛЮЧЕНИЕ 45

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 47

ВЕДЕНИЕ

Актуальность темы курсового проекта обусловлена возрастающей ролью автоматизированных систем сбора и обработки данных в современных программно-технических комплексах. В условиях непрерывного поступления телеметрической информации от внешних источников ручная обработка данных становится неэффективной и подверженной ошибкам, что затрудняет анализ состояния системы и принятие обоснованных решений. Отсутствие централизованного механизма приёма, хранения и визуализации телеметрии снижает оперативность контроля и увеличивает вероятность потери или искажения данных.

Современные клиент-серверные технологии позволяют организовать надёжный обмен телеметрическими данными по сетевым протоколам, автоматизировать их обработку и обеспечить удобный доступ к актуальной информации для пользователей. В связи с этим актуальной является разработка клиент-серверного приложения, обеспечивающего приём телеметрических данных по протоколу TCP, их обработку и наглядное представление в пользовательском интерфейсе.

Объектом курсового проекта является клиент-серверное приложение для сбора и обработки телеметрических данных.

Предметом исследования является процесс приёма, передачи, хранения и обработки телеметрической информации в клиент-серверной архитектуре.

Целью курсовой работы является разработка клиент-серверного приложения, обеспечивающего автоматизированный сбор, обработку и визуализацию телеметрических данных, поступающих от внешнего источника.

Для достижения поставленной цели решаются следующие задачи:

1. Выполнить анализ предметной области;
2. Провести сравнительный анализ существующих аналогов;
3. Определить оптимальную структуру системы;
4. Осуществить выбор средств реализации программного продукта;
5. Реализовать программные компоненты системы;
6. Разработать сопроводительную документацию.

1. АНАЛИЗ ТРЕБОВАНИЙ И ТЕХНОЛОГИЙ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

1.1. Основные требования к разрабатываемой системе

Требования к функциям, выполняемым системой

Разрабатываемая система должна предоставлять пользователям следующий функционал:

- Приём телеметрических данных от внешнего источника по протоколу TCP с последующей обработкой и хранением полученной информации;
- Обеспечение надёжного клиент-серверного взаимодействия между источником телеметрии, серверным приложением и клиентской частью системы;
- Обработка поступающих телеметрических данных с приведением их к структурированному виду, пригодному для дальнейшего анализа и отображения;
- Хранение актуальных телеметрических данных в памяти серверного приложения с возможностью получения последних значений по запросу клиента;
- Предоставление пользовательского интерфейса для отображения телеметрической информации в наглядной и удобной форме;
- Возможность просмотра текущих и последних полученных телеметрических данных, а также их основных параметров и состояний.

Требования к клиентской части системы

Клиентская часть разрабатываемой системы должна обеспечивать:

- адаптивный и интуитивно понятный пользовательский интерфейс, позволяющий удобно отображать и анализировать телеметрические данные;
- корректную обработку ошибок на стороне клиента с отображением уведомлений о сбоях и ошибках, возникающих на сервере;

- полную совместимость с REST API серверного приложения для получения и отображения актуальных телеметрических данных;
- безопасную работу с передаваемыми данными, а также защиту от распространённых уязвимостей клиентских приложений;
- достаточную производительность при обработке и отображении большого объёма телеметрической информации в режиме, близком к реальному времени.

Требования к серверной части системы

Серверная часть системы должна обладать следующими характеристиками:

- поддержка TCP-соединения для приёма телеметрических данных от внешнего источника;
- наличие REST API для взаимодействия с клиентским приложением и передачи обработанных данных;
- архитектура серверного приложения, основанная на монолитном подходе, в соответствии с требованиями курсового проекта;
- обработка исключительных и нештатных ситуаций с корректным оповещением клиентской части о возникших ошибках;
- высокая производительность при приёме, обработке и передаче телеметрических данных;
- возможность работы серверного приложения на операционных системах семейства Linux и Windows.

1.2. Анализ аналогов и прототипов

На сегодняшний день существует ряд программных решений, предназначенных для сбора, передачи и обработки телеметрических данных, однако универсальные клиент-серверные приложения, ориентированные на оперативный приём телеметрии от внешних источников по протоколу TCP с последующей

наглядной визуализацией и минимальными требованиями к настройке, представлены в недостаточном объёме. Многие существующие системы либо избыточно сложны, либо ориентированы на узкоспециализированные аппаратные комплексы, что затрудняет их использование в учебных и экспериментальных проектах.

PROMETHEUS

Рисунок 1.1 Система мониторинга «Prometheus»

Prometheus является широко распространённой системой мониторинга и сбора метрик. Данное решение ориентировано на работу с временными рядами и требует использования специализированного формата экспорта данных, а также дополнительной настройки компонентов сбора и визуализации. Использование Prometheus для приёма произвольных бинарных телеметрических данных по протоколу TCP требует значительной доработки и усложняет архитектуру системы. Кроме того, настройка и сопровождение системы может быть избыточной для задач оперативного анализа телеметрии в рамках курсового проекта.

ZABBIX

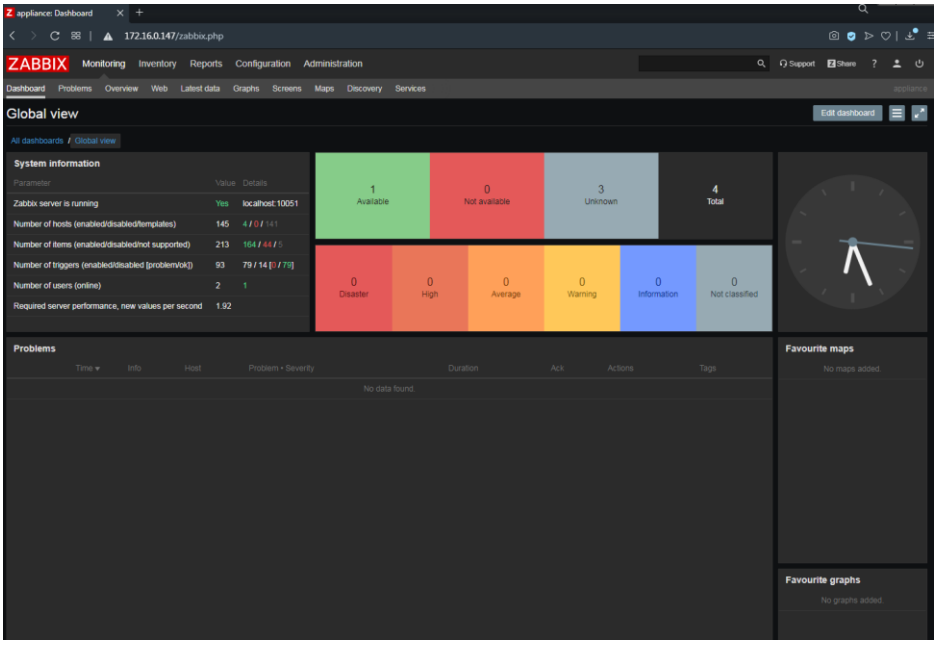


Рисунок 1.2 Система мониторинга «ZABBIX»

Система Zabbix представляет собой комплексное решение для мониторинга инфраструктуры и сетевых устройств. Она обладает широким функционалом, включающим оповещения, шаблоны и распределённую архитектуру, однако ориентирована преимущественно на мониторинг серверов и сетевого оборудования. Интерфейс системы содержит большое количество настроек и уровней конфигурации, что усложняет использование при необходимости быстрого приёма и отображения телеметрических данных от пользовательских источников. Кроме того, внедрение Zabbix требует использования базы данных и дополнительных сервисов, что увеличивает сложность развертывания и эксплуатации.

Таким образом, существующие решения либо обладают избыточным функционалом, либо не ориентированы на приём и обработку телеметрических данных в простом клиент-серверном приложении с использованием протокола TCP. В связи с этим разработка специализированного клиент-серверного приложения, обеспечивающего приём, обработку и визуализацию телеметрии с минимальными накладными расходами, является актуальной задачей.

1.3. Обоснование выбора инструментов и платформы для разработки клиентской части

Для разработки клиентской части приложения используются современные веб-технологии, обеспечивающие простоту реализации, высокую производительность и удобство визуализации телеметрических данных.

JavaScript (ES6+) — выбран в качестве основного языка разработки клиентской части приложения. Это обусловлено его широкой распространённостью, поддержкой всеми современными браузерами и наличием развитой экосистемы инструментов. Использование возможностей стандарта ES6+, таких как модули, стрелочные функции, асинхронные операции (async/await), деструктуризация и шаблонные строки, позволяет писать структурированный, читаемый и легко поддерживаемый код. JavaScript обеспечивает удобную реализацию логики взаимодействия с серверным REST API и обработки поступающих телеметрических данных.

HTML5 и **CSS3** — применяются для построения структуры пользовательского интерфейса и его визуального оформления. Использование современных возможностей HTML5 позволяет создавать адаптивные и семантически корректные веб-страницы, а CSS3 — реализовывать удобное и наглядное представление данных.

Данные технологии обеспечивают корректную работу клиентского приложения в различных браузерах без необходимости установки дополнительного программного обеспечения.

Canvas API — используется для визуализации телеметрических данных в графической форме. Данный подход позволяет отображать параметры ориентации и состояния системы в реальном времени, а также реализовывать наглядные графические элементы без использования сторонних библиотек. Canvas обеспечивает высокую производительность при частом обновлении изображения, что особенно важно при работе с потоковыми телеметрическими данными.

Fetch API / Axios — применяется для организации взаимодействия клиентской части с серверным REST API. Использование HTTP-клиента позволяет получать актуальные телеметрические данные, обрабатывать ответы сервера и корректно реагировать на возможные ошибки. Централизованная обработка запросов и ответов упрощает поддержку кода и повышает надёжность обмена данными между клиентом и сервером.

Выбор указанных технологий позволяет создать лёгкую, кроссбраузерную и производительную клиентскую часть, обеспечивающую удобное отображение и анализ телеметрической информации в рамках курсового проекта.

1.4. Обоснование выбора инструментов и платформы для разработки серверной части

Серверная часть разрабатываемого клиент-серверного приложения предназначена для приёма телеметрических данных от внешнего источника по протоколу TCP[4], их обработки, хранения в оперативной памяти и предоставления клиентской части через REST API. В связи с этим при выборе инструментов и платформы особое внимание уделялось производительности, надёжности сетевого взаимодействия и простоте архитектуры.

Обоснование выбора языков программирования и технологий

В качестве основного языка программирования серверной части выбран язык **Go (Golang)**. Данный язык обладает рядом преимуществ, которые делают его оптимальным для разработки сетевых и серверных приложений:

- высокая производительность и эффективная работа с памятью;
- встроенная поддержка конкурентности на основе горутин и каналов, что особенно важно при одновременной обработке нескольких TCP-соединений[4];
- развитая стандартная библиотека, включающая средства для работы с сетью, HTTP-серверами и сериализацией данных;

- простота синтаксиса и строгая типизация, повышающие надёжность и сопровождаемость кода;
- кроссплатформенность, позволяющая запускать серверное приложение на операционных системах Windows и Linux без изменений исходного кода.

Использование языка Go позволяет реализовать серверное приложение без привлечения сторонних фреймворков и дополнительных серверов приложений, что упрощает архитектуру и снижает накладные расходы.

Организация приёма телеметрических данных

Для приёма телеметрических данных используется **TCP-сервер**, реализованный средствами стандартной библиотеки Go. Протокол TCP выбран благодаря своей надёжности, гарантированной доставке данных и сохранению порядка передачи пакетов, что является критически важным при работе с телеметрической информацией.

Серверная часть принимает бинарные кадры телеметрии, выполняет их разбор, проверку корректности и преобразование в структурированный формат, пригодный для дальнейшей обработки и отображения.

Хранение данных

В рамках курсового проекта для хранения телеметрических данных используется **оперативная память сервера**. Такой подход обусловлен следующими причинами:

- высокая скорость доступа к данным;
- отсутствие необходимости в настройке и сопровождении внешней СУБД;
- соответствие требованиям учебного проекта, ориентированного на демонстрацию принципов клиент-серверного взаимодействия и обработки потоковых данных.

Хранение данных в памяти позволяет оперативно предоставлять клиентской части актуальные значения телеметрии без дополнительных задержек, связанных с обращением к базе данных.

Реализация REST API

Для взаимодействия с клиентской частью серверное приложение предоставляет **REST API**, реализованное с использованием стандартного HTTP-пакета языка Go. REST API обеспечивает:

- получение актуальных телеметрических данных по запросу клиента;
- передачу данных в формате JSON, удобном для обработки на стороне клиентского приложения;
- централизованную обработку ошибок и исключительных ситуаций.

Использование REST-подхода обеспечивает слабую связанность клиентской и серверной частей и упрощает возможное расширение системы в дальнейшем.

Архитектура серверного приложения

Серверная часть реализована в виде **монолитного приложения**, что соответствует требованиям курсового проекта. Внутренняя структура сервера логически разделена на следующие модули:

- модуль приёма данных (TCP-сервер);
- модуль обработки телеметрии;
- модуль хранения данных в памяти;
- модуль REST API для взаимодействия с клиентом.

Данное разделение повышает читаемость кода и упрощает его сопровождение, при этом не усложняя общую архитектуру приложения.

Выводы

Таким образом, исходя из требований к разрабатываемому клиент-серверному приложению для приёма, обработки и отображения телеметрических данных, а также с учётом особенностей архитектуры системы и проведённого анализа современных инструментов разработки, был выбран следующий технологический стек.

Клиентская часть приложения реализуется с использованием стандартных веб-технологий **HTML5, CSS3 и JavaScript (ES6+)**, что обеспечивает кросс-браузерность, простоту развёртывания и удобную визуализацию телеметрической информации без необходимости установки дополнительного программного обеспечения. Использование Canvas API позволяет наглядно отображать параметры состояния системы в режиме, близком к реальному времени.

Серверная часть приложения реализуется на языке программирования **Go**, который выбран благодаря высокой производительности, встроенной поддержке конкурентного выполнения, развитым сетевым возможностям и кроссплатформенности. Для приёма телеметрических данных используется протокол **TCP**, обеспечивающий надёжную передачу информации от внешнего источника.

Взаимодействие клиентской и серверной частей организовано посредством **REST API**, что обеспечивает слабую связанность компонентов системы, удобство расширения функциональности и простоту интеграции с клиентским интерфейсом. Хранение данных осуществляется в оперативной памяти сервера, что позволяет минимизировать задержки при доступе к актуальной телеметрической информации и соответствует требованиям учебного проекта.

Выбор указанных технологий полностью соответствует задачам курсового проекта, обеспечивает необходимую производительность, надёжность и наглядно демонстрирует принципы построения клиент-серверных приложений и обработки потоковых телеметрических данных.

2. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1. Разработка структуры системы

Разрабатываемое клиент-серверное приложение предназначено для приёма, обработки и визуализации телеметрических данных от внешнего источника в режиме реального времени. Система состоит из следующих основных компонентов:

Клиентская часть — реализована с использованием веб-технологий **HTML5, CSS3 и JavaScript (ES6+)** с применением Canvas API для визуализации телеметрических параметров и библиотеки **Axios** для взаимодействия с сервером через REST API. Клиент предназначен для работы пользователей на персональных компьютерах или планшетах и обеспечивает:

- наглядное отображение текущих значений телеметрии;
- обработку ошибок при взаимодействии с сервером и уведомление пользователя;
- адаптивный и интуитивно понятный интерфейс;
- возможность быстрого обновления и доработки функционала без остановки сервиса.

Серверная часть — полностью реализована на языке **Go**, что позволяет обеспечивать высокую производительность при одновременной обработке нескольких TCP-соединений. Сервер выполняет следующие функции:

- приём телеметрических данных по протоколу TCP;
- проверку корректности и структурирование поступающих данных;
- хранение актуальной информации в оперативной памяти;
- предоставление клиентской части актуальных значений через REST API;
- обработку исключительных ситуаций и уведомление клиента о возникших ошибках;
- кроссплатформенное исполнение на ОС Windows и Linux.

Механизм обмена данными — реализован через REST API, предоставляющее клиенту методы для получения актуальной телеметрии, управления сессиями и подтверждения успешной обработки данных. Все обмены выполняются в формате JSON с поддержкой заголовков авторизации и контролем целостности передаваемых данных.

Предложенная архитектура обеспечивает:

- надёжность и производительность при приёме и обработке телеметрических данных;
- простоту сопровождения и расширения функционала без изменения базовой архитектуры;
- возможность визуализации информации в реальном времени для удобства анализа.

Организация хранения данных в оперативной памяти

В рамках разрабатываемого клиент-серверного приложения для приёма и обработки телеметрических данных внешняя система управления базами данных не используется. Хранение информации осуществляется в оперативной памяти серверного приложения, что позволяет обеспечить минимальные задержки при доступе к данным и упростить архитектуру системы.

Для хранения телеметрии на сервере реализован специализированный модуль памяти, представляющий собой потокобезопасное хранилище[13], рассчитанное на хранение ограниченного количества последних полученных кадров телеметрии. Такой подход соответствует требованиям проекта, ориентированного на обработку и визуализацию актуальных данных в режиме, близком к реальному времени.

Хранилище реализовано в виде структуры **MemoryStore**, которая содержит:

- массив телеметрических кадров;

- параметр, определяющий максимальное количество хранимых записей;
- механизм взаимного исключения для обеспечения потокобезопасного доступа.

При поступлении нового кадра телеметрии он добавляется в конец массива. В случае превышения допустимого размера хранилища наиболее старые данные автоматически удаляются. Это позволяет поддерживать фиксированный объём используемой памяти и предотвращает её неконтролируемый рост при длительной работе сервера.

Для обеспечения корректной работы в условиях многопоточной обработки данных доступ к хранилищу синхронизируется с использованием мьютекса, что исключает состояния гонки при одновременном добавлении и чтении данных.

Хранилище предоставляет следующие основные операции:

- добавление нового кадра телеметрии;
- получение всей истории сохранённых данных;
- получение последнего (актуального) кадра телеметрии.

Структура телеметрических данных

Телеметрические данные, поступающие на сервер по протоколу TCP, представляют собой бинарные пакеты фиксированного формата. Для их обработки и дальнейшего использования в системе реализована структура Telemetry, описывающая набор параметров состояния объекта.

Структура телеметрических данных включает следующие поля:

- время работы бортовой системы;
- значение скорости;
- углы ориентации объекта (крен, тангаж, рысканье);

- среднюю температуру, а также температуры со стороны освещения и тени;
- географические координаты (широта и долгота);
- высоту над уровнем моря.

Каждый принятый бинарный пакет преобразуется в экземпляр структуры **Telemetry** с использованием побайтового чтения в формате Little Endian, что обеспечивает корректную интерпретацию данных, передаваемых внешним источником.

После успешного разбора телеметрический кадр передаётся в модуль хранения данных и становится доступным для получения через REST API серверного приложения и последующего отображения в клиентском интерфейсе.

Выбранный формат представления телеметрических данных является компактным, легко расширяемым и удобным для сериализации в формат JSON при передаче данных клиентской части системы.

2.2. Структура телеметрического кадра

В разрабатываемом клиент-серверном приложении основным объектом обработки и хранения данных является **телеметрический кадр**, представляющий собой бинарный пакет фиксированной структуры, передаваемый от внешнего источника на сервер по протоколу TCP.

Каждый телеметрический кадр содержит набор параметров, описывающих текущее состояние объекта, и передаётся в виде последовательности байтов в формате **Little Endian**. Фиксированный формат кадра позволяет обеспечить высокую скорость передачи и минимальные накладные расходы при обработке данных.

Структура телеметрического кадра представлена в таблице:

Наименование поля	Описание	Тип данных	Размер, байт
OnboardTime	Бортовое время	float64	8
Velocity	Скорость объекта	float32	4

Roll	Угол крена	float32	4
Pitch	Угол тангажа	float32	4
Yaw	Угол рысканья	float32	4
TempAvg	Средняя температура	float32	4
TempSun	Температура освещённой стороны	float32	4
TempShadow	Температура теневой стороны	float32	4
Latitude	Географическая широта	float32	4
Longitude	Географическая долгота	float32	4
Altitude	Высота над уровнем моря	float32	4

Подробное описание полей кадра

1. OnboardTime

- **Тип:** float64
- **Размер:** 8 байт
- **Описание:** Время на атомных часах на борту аппарата. Используется для синхронизации телеметрии и анализа временных зависимостей параметров.
- **Единицы измерения:** секунды

2. Velocity

- **Тип:** float32
- **Размер:** 4 байта
- **Описание:** Линейная скорость движения объекта в текущий момент времени.
- **Единицы измерения:** м/с

3. Roll

- **Тип:** float32

- **Размер:** 4 байта
- **Описание:** Угол крена — вращение объекта вокруг продольной оси.
- **Единицы измерения:** градусы

4. Pitch

- **Тип:** float32
- **Размер:** 4 байта
- **Описание:** Угол тангажа — вращение объекта вокруг поперечной оси.
- **Единицы измерения:** градусы

5. Yaw

- **Тип:** float32
- **Размер:** 4 байта
- **Описание:** Угол рыскания — вращение объекта вокруг вертикальной оси.
- **Единицы измерения:** градусы

6. TempAvg

- **Тип:** float32
- **Размер:** 4 байта
- **Описание:** Средняя температура объекта, вычисляемая на основе данных датчиков.
- **Единицы измерения:** °C

7. TempSun

- **Тип:** float32
- **Размер:** 4 байта
- **Описание:** Температура поверхности объекта, обращённой к источнику излучения (например, Солнцу).
- **Единицы измерения:** °C

8. TempShadow

- **Тип:** float32
- **Размер:** 4 байта

- **Описание:** Температура затенённой стороны объекта.
- **Единицы измерения:** °C

9. Latitude

- **Тип:** float32
- **Размер:** 4 байта
- **Описание:** Географическая широта положения объекта.
- **Единицы измерения:** градусы

10. Longitude

- **Тип:** float32
- **Размер:** 4 байта
- **Описание:** Географическая долгота положения объекта.
- **Единицы измерения:** градусы

11. Altitude

- **Тип:** float32
- **Размер:** 4 байта
- **Описание:** Высота объекта относительно уровня моря.
- **Единицы измерения:** метры

Общий размер телеметрического кадра составляет 44 байта.

Использование бинарного формата передачи данных позволяет значительно снизить объём передаваемой информации по сравнению с текстовыми форматами, такими как JSON или XML, что особенно важно при передаче телеметрии в режиме реального времени.

После приёма телеметрического кадра серверное приложение выполняет его побайтовый разбор и преобразует данные в структурированный вид для дальнейшего хранения в оперативной памяти и предоставления клиентской части через REST API. При передаче клиенту телеметрические данные сериализуются в формат JSON, удобный для обработки и визуализации в веб-интерфейсе.

Фиксированная структура кадра упрощает расширение системы: при необходимости добавления новых параметров формат может быть дополнен новыми полями с сохранением общей логики обработки данных.

2.3. Визуализация структуры проекта

Ниже представлены таблица структуры кадра телеметрии, диаграмма компонентов, а также диаграмма классов серверной части проекта.

Offset	Field	Type	Size
0	OnboardTime	float64	8 B
8	Velocity	float32	4 B
12	Roll	float32	4 B
16	Pitch	float32	4 B
20	Yaw	float32	4 B
24	TempAvg	float32	4 B
28	TempSun	float32	4 B
32	TempShadow	float32	4 B
36	Latitude	float32	4 B
40	Longitude	float32	4 B
44	Altitude	float32	4 B

Рисунок 2.1 Таблица структуры кадра телеметрии

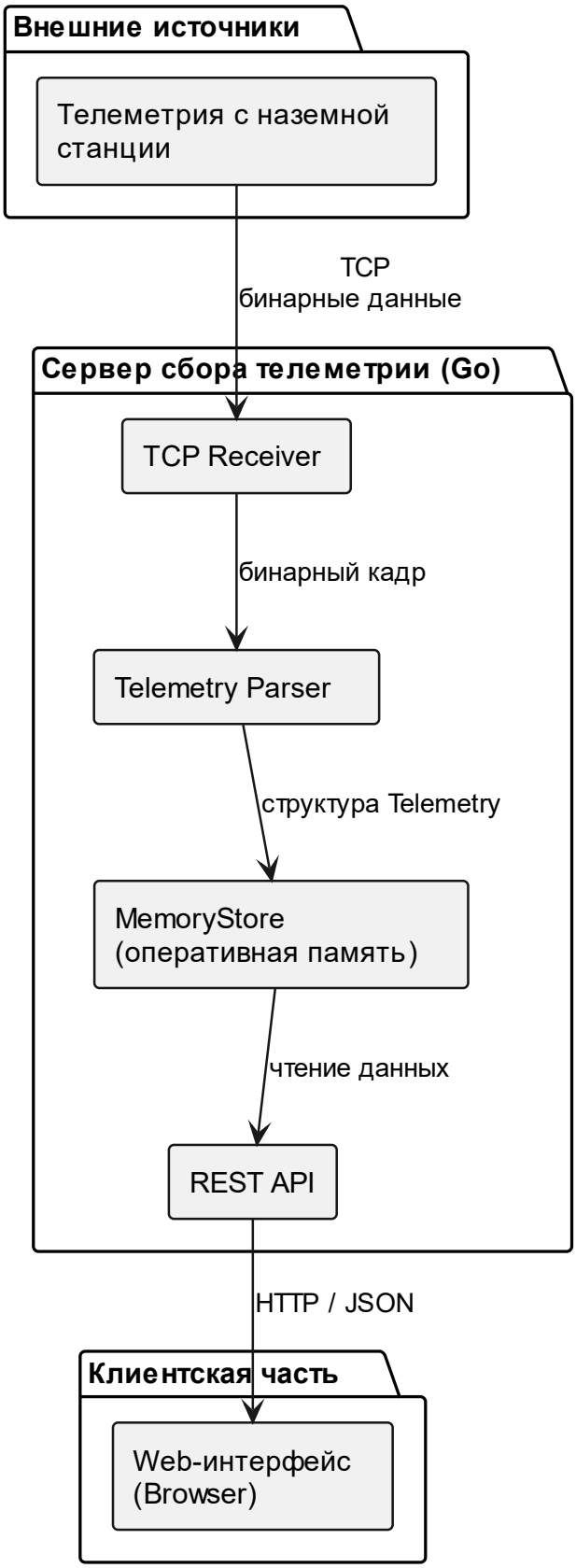


Рисунок 2.2 Диаграмма компонентов проекта

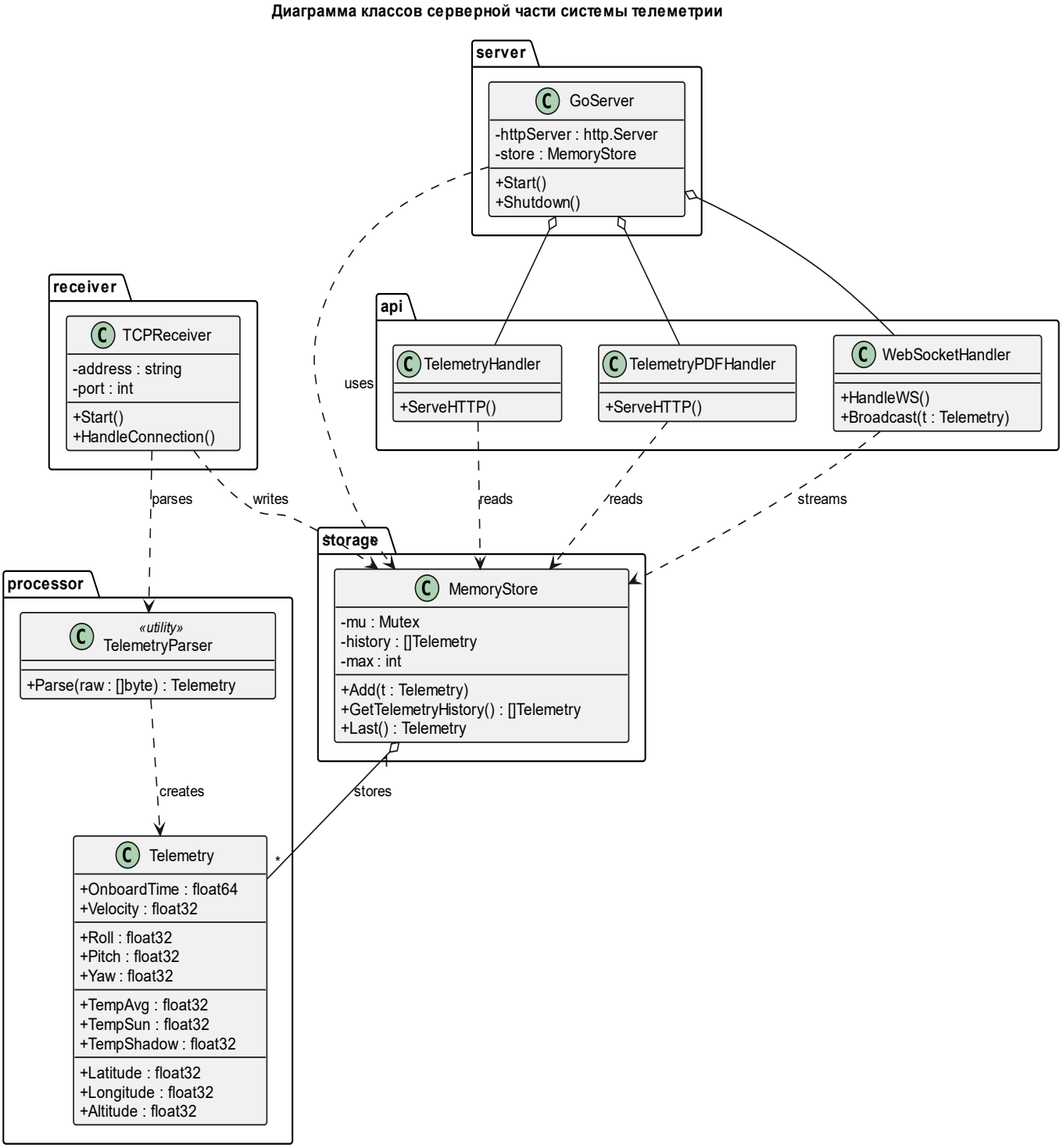
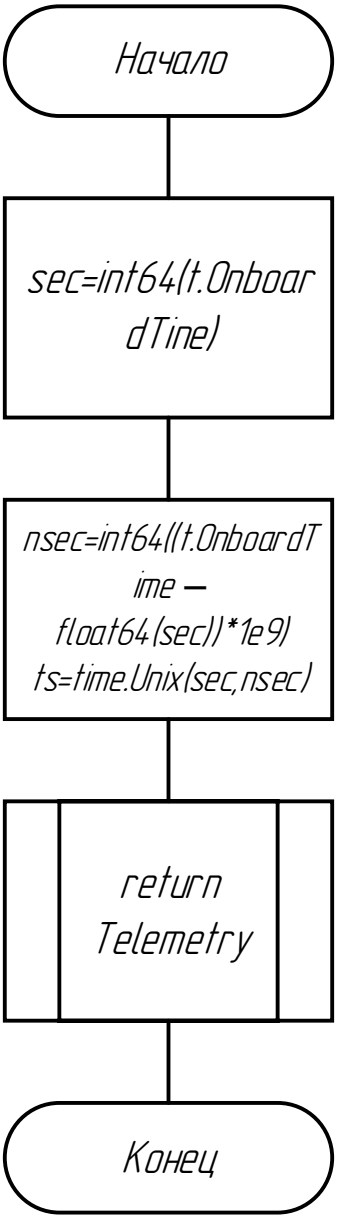


Рисунок 2.3 Диаграмма классов серверной части

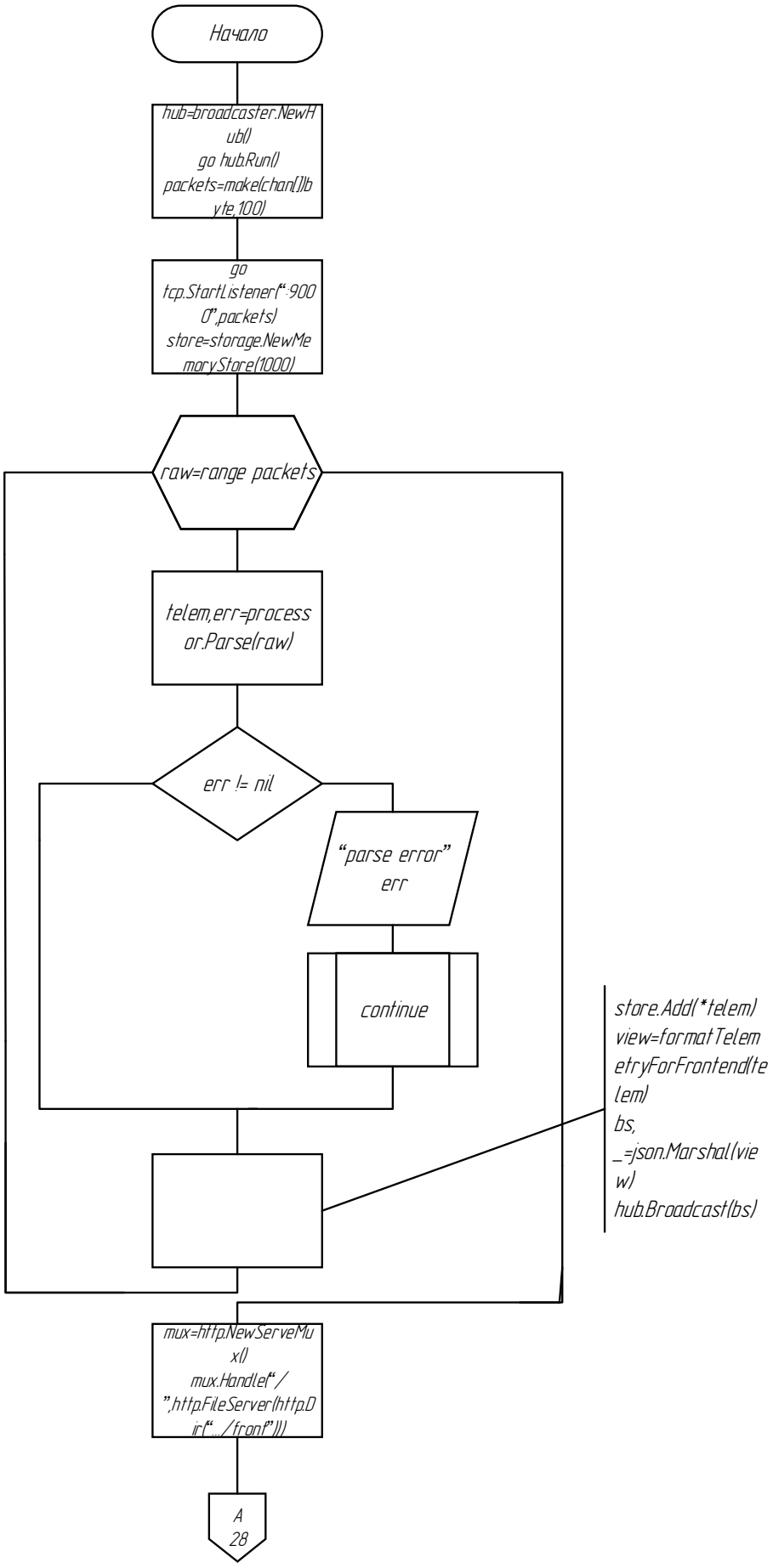
2.4 Блок-схемы основных алгоритмов проекта

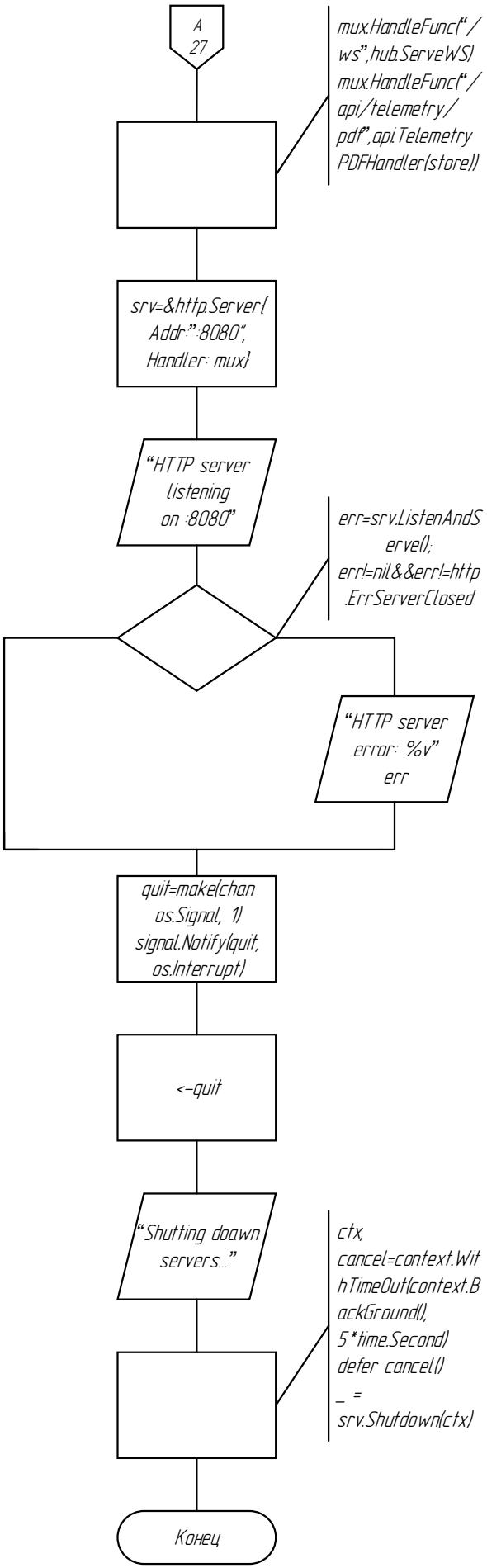
<u>Файл main.go:</u>

Функция **formatTelemetryForFrontend(t *processor.Telemetry) map[string]interface{}:**



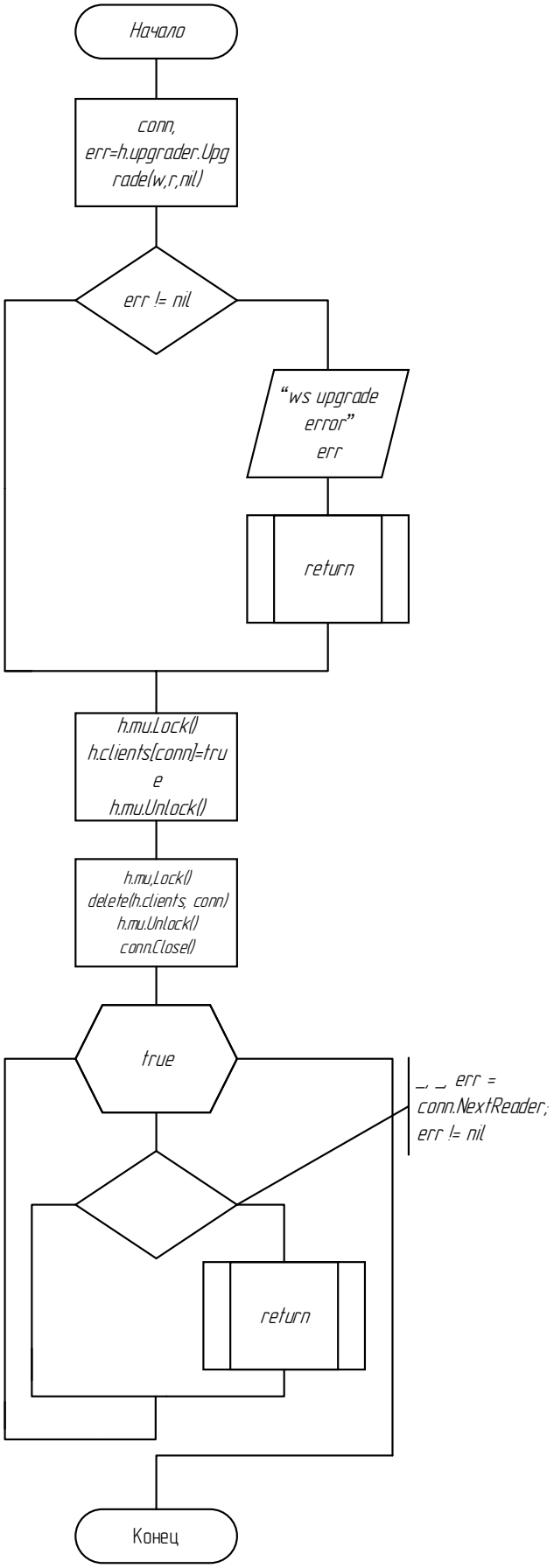
Функция **main():**



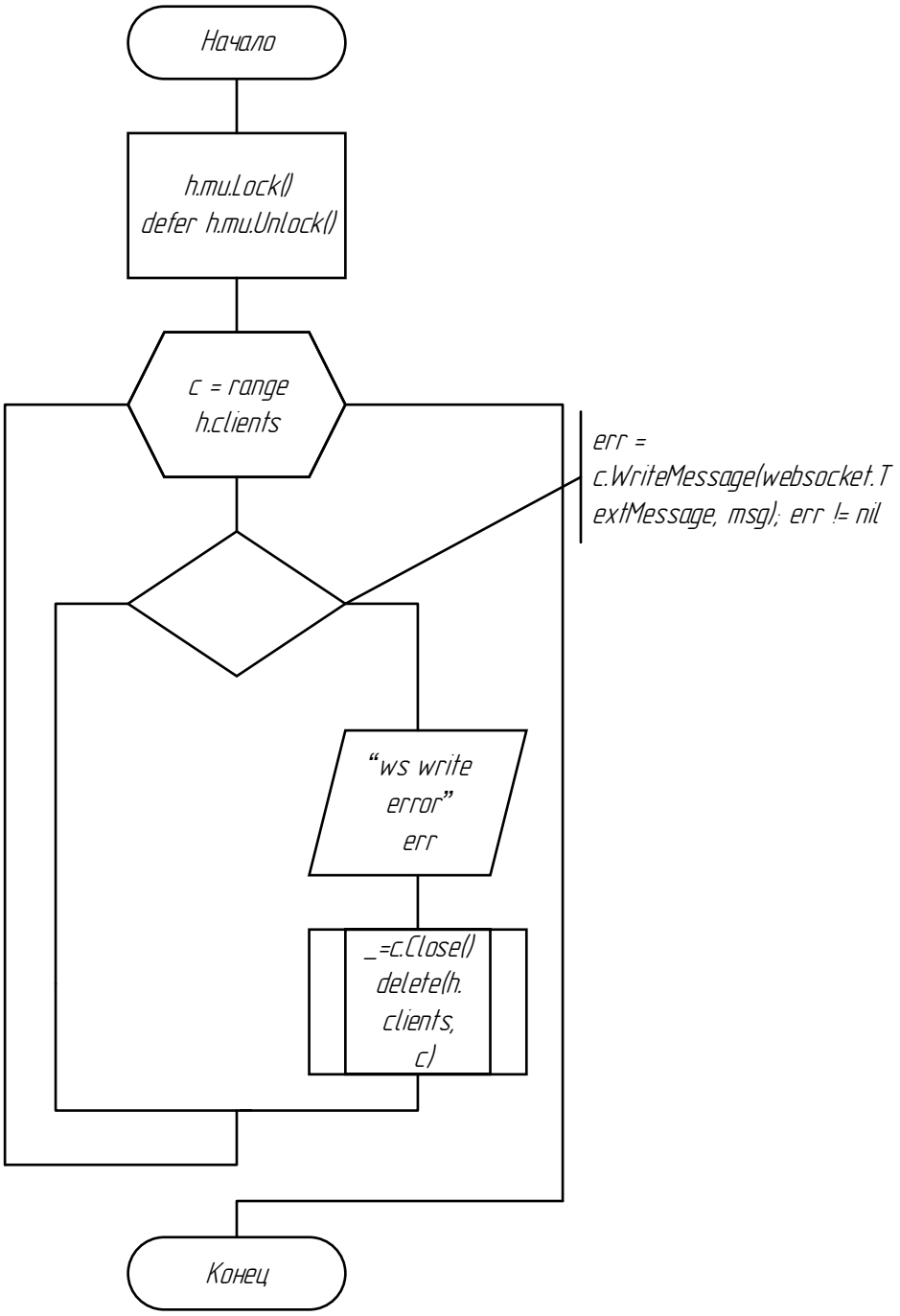

A
27
mux.HandleFunc("/ws",hub.ServeWS)
mux.HandleFunc("/api/telemetry/pdf",apiTelemetryPDFHandler(store))
srv=&http.Server{Addr:":8080", Handler: mux}
"HTTP server listening on :8080"
err=srv.ListenAndServe();
err!=nil&&err!=http.ErrServerClosed
"HTTP server error: %v" err
quit=make(chan os.Signal, 1)
signal.Notify(quit, os.Interrupt)
<-quit
"Shutting doawn servers..."
ctx, cancel=context.WithTimeOut(context.BackGround(), 5*time.Second)
defer cancel()
_ = srv.Shutdown(ctx)
Конец

<u>Файл ws_broadcaster.go:</u>

Функция **ServeWS(w http.ResponseWriter, r *http.Request):**

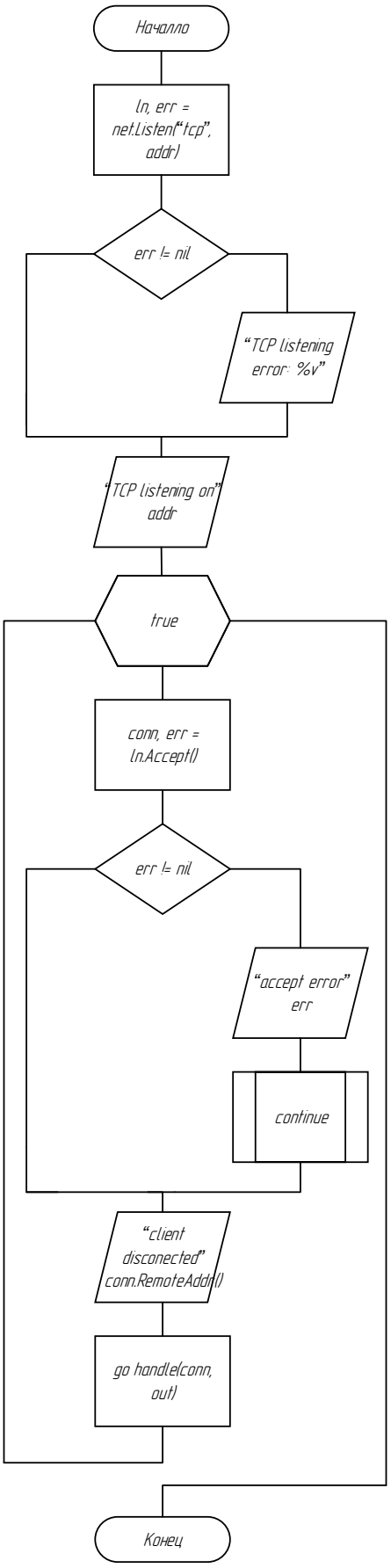


Функция **Broadcast(msg []byte):**

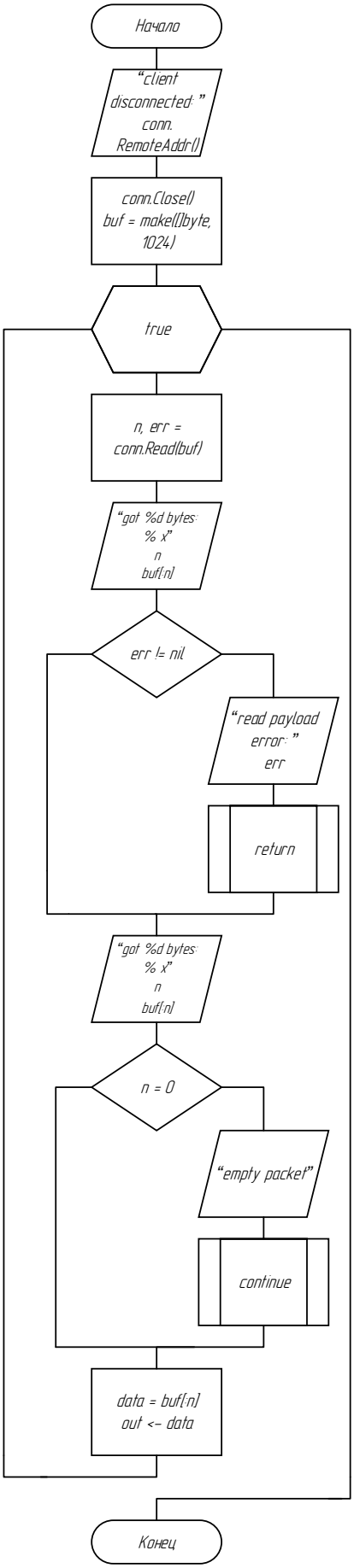


<u>Файл tcp_receiver.go:</u>

Функция **StartListener(addr string, out chan<- []byte):**



Функция **handle(conn net.Conn, out chan<- []byte):**



Выводы

Таким образом, был спроектирован программный продукт, реализующий клиент-серверную архитектуру и предназначенный для приёма, обработки, хранения и визуализации телеметрических данных в режиме реального времени. Система состоит из серверной части, реализованной на языке программирования Go, и клиентской части, обеспечивающей отображение получаемой телеметрии. Все компоненты системы были спроектированы и реализованы в соответствии с функциональными требованиями технического задания.

Хранение данных в разрабатываемом приложении осуществляется **в оперативной памяти сервера** без использования системы управления базами данных. Для этого используется специализированное in-memory-хранилище, реализованное в виде структуры MemoryStore, обеспечивающей потокобезопасное добавление, хранение и чтение телеметрических кадров. Такой подход позволяет минимизировать задержки при обработке данных и обеспечивает высокую производительность системы при непрерывном поступлении телеметрии.

Серверная часть приложения реализована на языке программирования Go и включает в себя несколько логически разделённых модулей: модуль приёма бинарных данных по протоколу TCP, модуль обработки и парсинга телеметрических кадров, модуль хранения данных в памяти, а также HTTP- и WebSocket-сервер для взаимодействия с клиентской частью. Использование языка Go обусловлено его высокой производительностью[10], эффективной работой с сетевыми соединениями, встроенной поддержкой конкурентности и возможностью создания надёжных серверных приложений.

Приём телеметрических данных осуществляется по TCP-соединению в виде бинарных кадров фиксированной структуры. Полученные данные передаются в модуль обработки, где выполняется их декодирование и преобразование в структуру Telemetry. После этого данные сохраняются в оперативном хранилище и становятся доступными для дальнейшего использования.

Для взаимодействия с клиентской частью реализован HTTP-сервер, предоставляющий REST-интерфейсы для получения текущих и исторических данных, а также WebSocket-соединение для передачи телеметрии в режиме реального времени. Использование WebSocket позволяет передавать данные клиенту без необходимости постоянного опроса сервера, что снижает сетевую нагрузку и повышает отзывчивость системы.

В результате был разработан устойчивый и расширяемый программный продукт с чётким разделением ответственности между компонентами. Выбранная архитектура и используемые технологии обеспечивают высокую производительность, простоту сопровождения и возможность дальнейшего развития системы, включая добавление постоянного хранилища, масштабирование или расширение функциональности обработки телеметрических данных.

3. КОНТРОЛЬ КАЧЕСТВА И ИНТЕГРАЦИЯ КОМПОНЕНТОВ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

3.1. Интерфейс клиентской части

Интерфейс клиентской части веб-приложения был разработан с использованием библиотеки React.js, что позволило реализовать интерактивный, наглядный и современный пользовательский интерфейс для визуализации телеметрических данных. При проектировании интерфейса основное внимание уделялось простоте навигации, высокой информативности и минимальной когнитивной нагрузке на пользователя, что особенно важно при работе с потоковыми данными в режиме реального времени.

Клиентское приложение построено на компонентном подходе, при котором интерфейс формируется из независимых и переиспользуемых элементов, отвечающих за отображение отдельных параметров телеметрии, графиков и текущего состояния системы. Такой подход упрощает сопровождение и дальнейшее расширение функциональности приложения.

Интерфейс обеспечивает отображение текущих значений телеметрических параметров, а также их динамики во времени, получаемых от серверной части через HTTP- и WebSocket-соединения. Реализована автоматическая актуализация данных без перезагрузки страницы, что повышает удобство работы и позволяет оперативно отслеживать изменения параметров.

Веб-приложение адаптировано для работы на различных устройствах и разрешениях экранов, что обеспечивает корректное отображение интерфейса как на настольных компьютерах, так и на мобильных устройствах. Использование современных веб-технологий позволило создать отзывчивый и производительный пользовательский интерфейс, полностью соответствующий требованиям проекта.

Параметр	Значение
Бортовое время, с	28.12.2025 18:42:56.629
Скорость, км/с	7.67
Крен, °	-0.02
Тангаж, °	-0.25
Рысканье, °	6.00
Средняя температура, °C	20.08
Температура освещённой стороны, °C	44.94
Температура неосвещённой стороны, °C	0.18
Широта	15.25
Долгота	-162.81
Высота, км	400.00

Рисунок 3.1 Таблица с пориходящей телеметрией

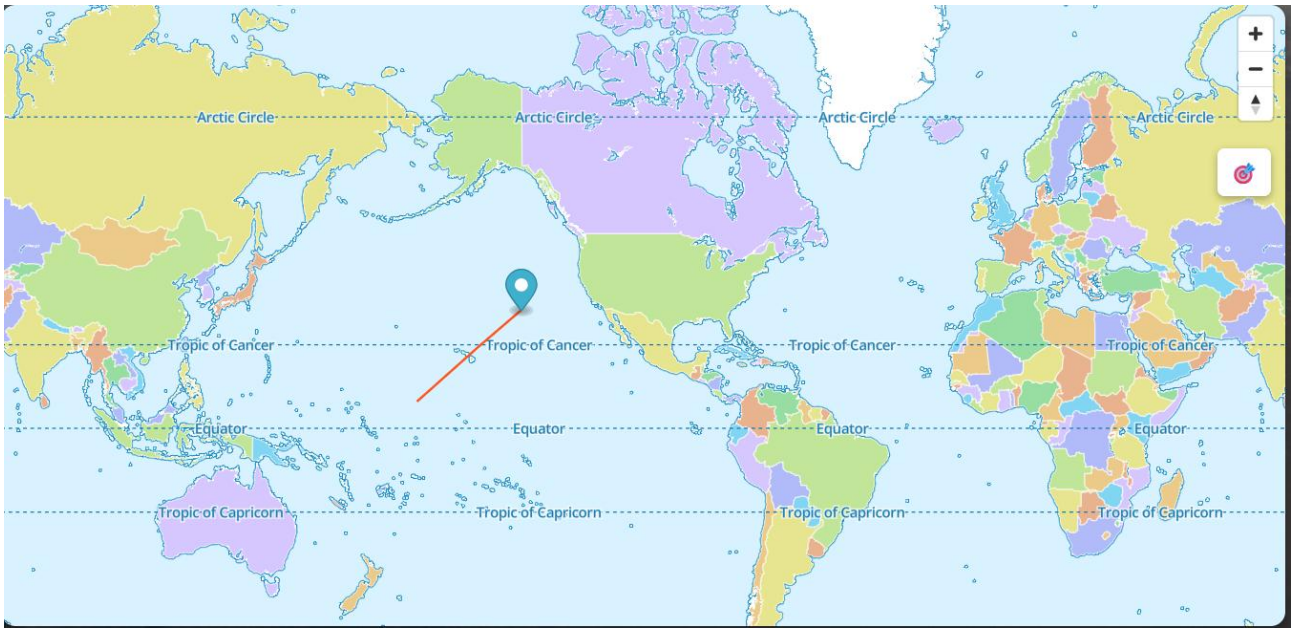


Рисунок 3.2 Местоположение аппарата на карте

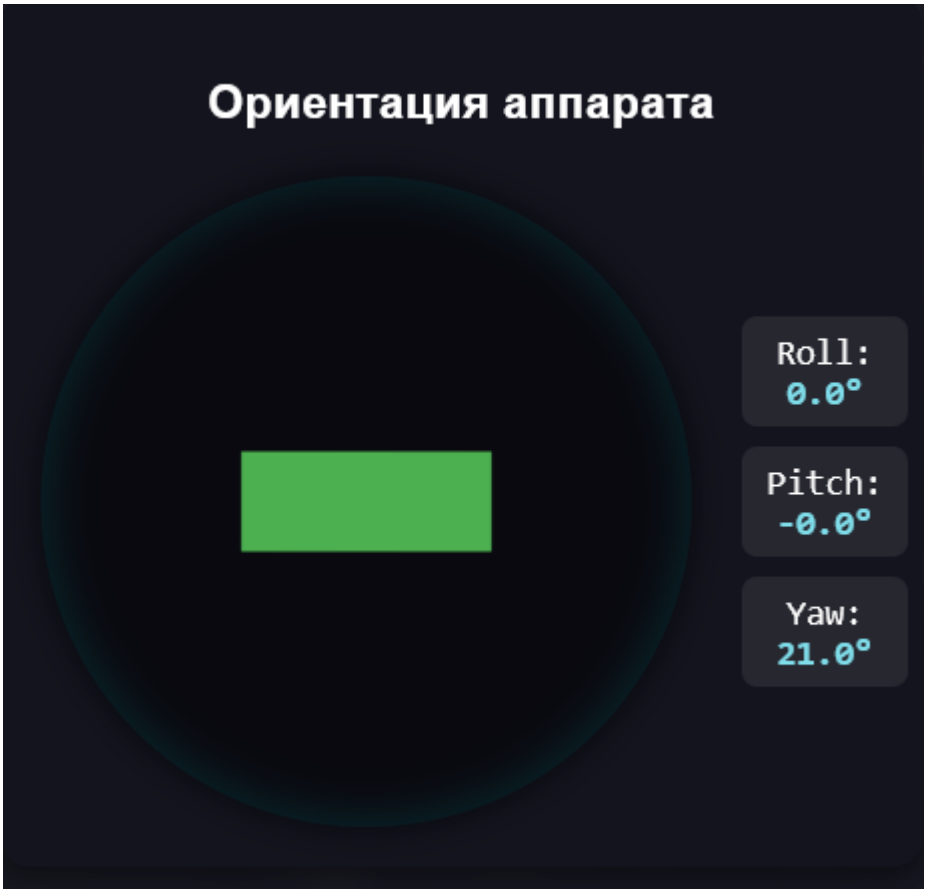


Рисунок 3.3 График поворотной ориентации аппарата

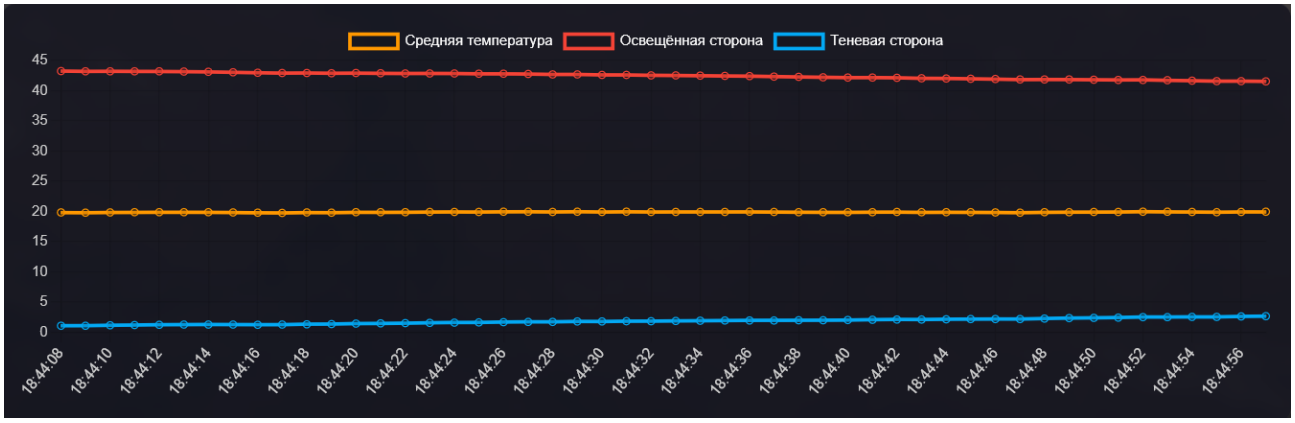


Рисунок 3.4 График изменения температуры аппарата

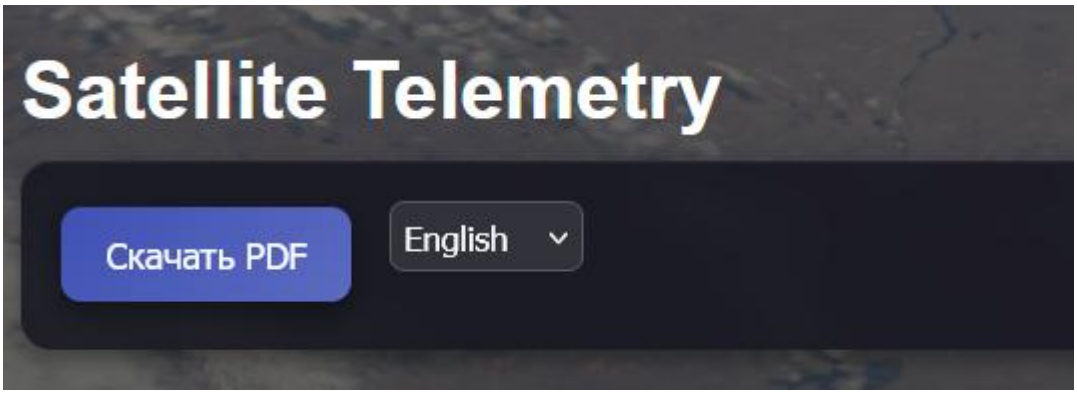


Рисунок 3.5 Кнопки изменения языка и скачивания журнала сеансов связи

Сеанс №1

Время: 28.12.2025 18:46:08.816

Скорость: 7.67 км/с

Ориентация (R/P/Y): 0.21 / -0.24 / 44.40

Температуры: средняя 20.2, солнце 38.1, тень 5.9

Координаты: 41.1028, -52.8034, высота 400.0 км

Сеанс №2

Время: 28.12.2025 18:46:09.817

Скорость: 7.67 км/с

Ориентация (R/P/Y): 0.24 / -0.26 / 44.60

Температуры: средняя 20.2, солнце 38.0, тень 6.0

Координаты: 40.7888, -52.2304, высота 400.0 км

Сеанс №3

Время: 28.12.2025 18:46:10.818

Скорость: 7.67 км/с

Ориентация (R/P/Y): 0.24 / -0.23 / 44.80

Температуры: средняя 20.2, солнце 38.0, тень 6.0

Координаты: 40.4707, -51.6575, высота 400.0 км

Сеанс №4

Рисунок 3.6 Структура журнала телеметрии в формате pdf

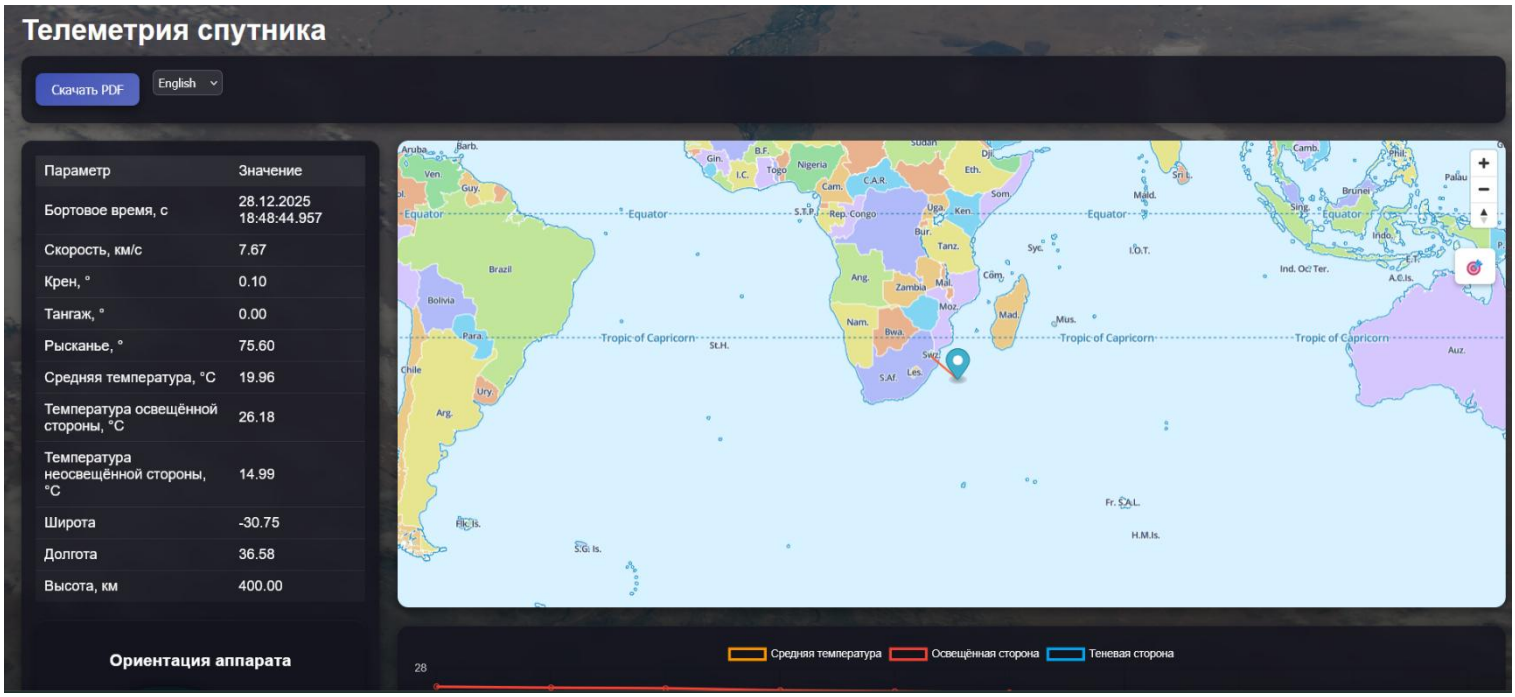


Рисунок 3.7 Общий вид страницы телеметрии

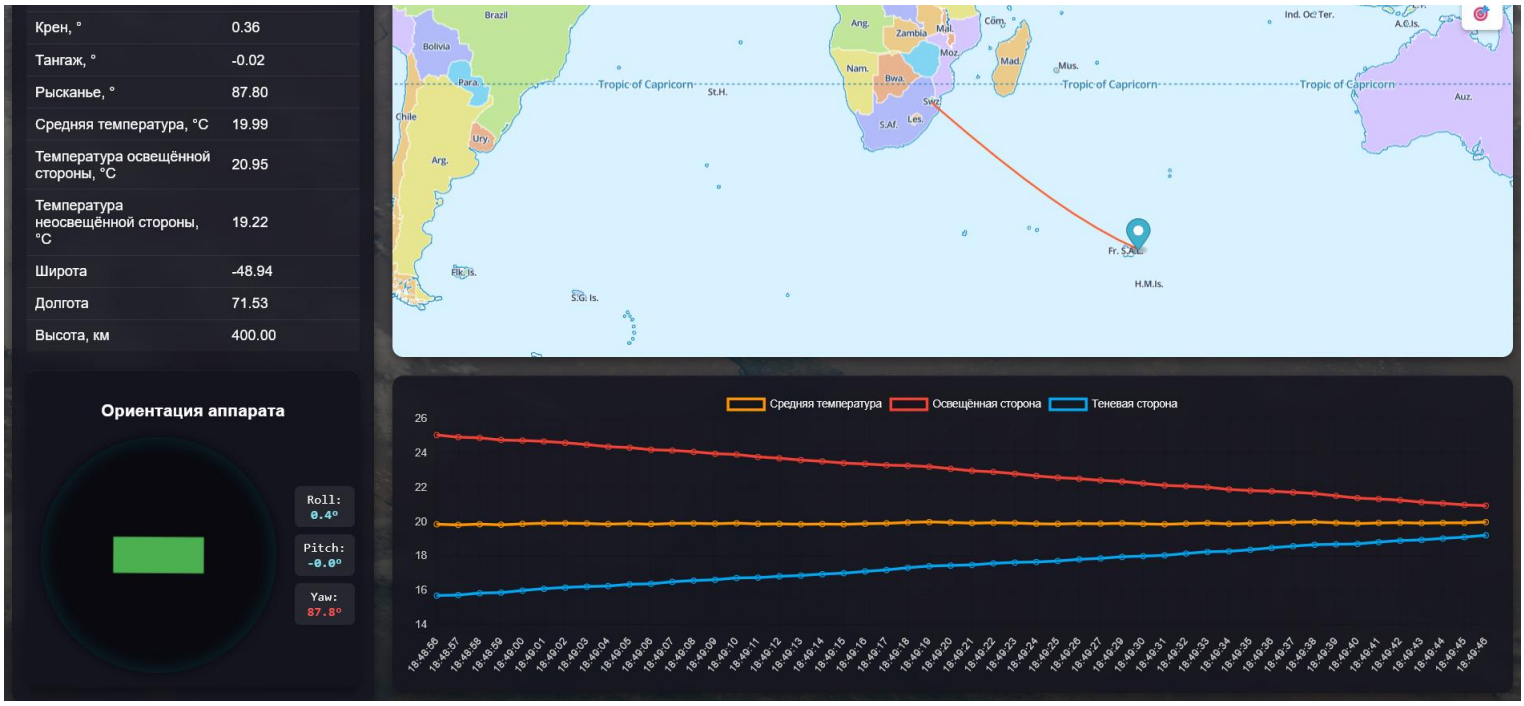


Рисунок 3.8 Общий вид страницы телеметрии

3.2. Пользовательский интерфейс клиентской части

Таким образом, интерфейс клиентской части веб-приложения обеспечивает пользователю быстрый и наглядный доступ к данным телеметрии, поступающим от серверной части системы. Благодаря компонентной архитектуре React.js, интуитивно понятной навигации и минималистичному дизайну, пользовательский интерфейс ориентирован на оперативное восприятие информации и удобство работы с потоковыми данными в режиме реального времени.

Интерфейс приложения построен с использованием независимых визуальных компонентов, отвечающих за отображение отдельных телеметрических параметров, графиков и текущего состояния системы. Обновление данных происходит автоматически без перезагрузки страницы, что позволяет пользователю наблюдать изменение параметров в реальном времени. Минимальное количество элементов управления и логичное расположение информации обеспечивают простоту использования и высокую читаемость интерфейса.

Скриншоты веб-приложения наглядно демонстрируют основные экраны отображения телеметрии, включая текущие значения параметров, графики изменения величин во времени и элементы управления визуализацией данных, что подтверждает функциональность и удобство разработанной клиентской части.

3.3. Руководство пользователя

Для работы с приложением пользователю необходимо иметь устройство с современным веб-браузером и доступ к серверной части системы. Установка дополнительного программного обеспечения на стороне клиента не требуется.

После открытия веб-приложения пользователь автоматически попадает на главную страницу, на которой отображаются текущие телеметрические данные, получаемые от сервера. Данные обновляются в режиме реального времени с использованием WebSocket-соединения, что позволяет отслеживать изменения параметров без задержек.

На главном экране пользователю доступны:

- текущие значения основных телеметрических параметров (время на борту, скорость, ориентация, температура, координаты);
- графики изменения параметров во времени;
- индикаторы состояния соединения с сервером.

При необходимости пользователь может просмотреть историю ранее полученных телеметрических данных, сохранённых в оперативной памяти сервера, а также переключаться между различными режимами отображения информации. Интерфейс приложения адаптирован под различные разрешения экранов и корректно работает как на настольных компьютерах, так и на мобильных устройствах.

3.4. Руководство администратора

Для запуска и корректной работы серверной части приложения необходимо выполнить ряд последовательных действий, обеспечивающих приём, обработку и передачу телеметрических данных клиентам.

1. **Запуск серверного приложения**

   Серверная часть запускается на компьютере или сервере под управлением операционной системы Windows или Linux. После запуска Go-сервер начинает прослушивание TCP-порта для приёма бинарных телеметрических кадров, а также HTTP- и WebSocket-портов для взаимодействия с клиентской частью.

2. **Настройка параметров сервера**

   В конфигурационных файлах или переменных окружения задаются параметры сетевых подключений, включая адреса и порты TCP-, HTTP- и WebSocket-серверов, а также максимальный размер истории телеметрических данных, хранимых в оперативной памяти.

3. **Подключение источника телеметрии**

   Внешний источник телеметрических данных (генератор или устройство) подключается к серверу по TCP-протоколу и передаёт бинарные кадры

фиксированной структуры. Сервер автоматически выполняет их приём, декодирование и сохранение в оперативном хранилище.

4. **Подключение клиентских приложений**

   После запуска сервера пользователи могут подключаться к веб-приложению через браузер. WebSocket-соединение[8] обеспечивает передачу телеметрических данных в режиме реального времени всем подключённым клиентам.

5. **Контроль работы системы**

   Администратор может контролировать корректность работы системы по журналам сервера, состоянию сетевых соединений и отображению данных в клиентском интерфейсе. Архитектура приложения позволяет при необходимости расширить функциональность, добавив постоянное хранилище данных, авторизацию пользователей или дополнительные методы обработки телеметрии.

Выводы

В результате выполнения курсового проекта был реализован полный цикл разработки клиент-серверного приложения для приёма, обработки и визуализации телеметрических данных в режиме реального времени. В ходе работы были выполнены анализ требований, проектирование архитектуры системы, разработка серверной и клиентской частей, а также тестирование и развёртывание программного продукта.

Разработанное серверное приложение, реализованное на языке программирования Go, обеспечивает приём бинарных телеметрических кадров по TCP-протоколу, их декодирование и сохранение в оперативной памяти с использованием потокобезопасного in-memory-хранилища. Для взаимодействия с клиентской частью реализованы HTTP- и WebSocket-интерфейсы, позволяющие получать как текущие, так и исторические данные, а также передавать телеметрию в режиме

реального времени[8]. Проведённое тестирование подтвердило корректность обработки входящих данных, стабильность работы сервера и отсутствие ошибок при непрерывной передаче телеметрии.

Клиентская часть веб-приложения, разработанная с использованием библиотеки React.js, обеспечивает наглядное отображение телеметрических параметров и их динамики во времени. Интерфейс приложения обновляется автоматически без перезагрузки страницы и корректно работает на различных устройствах и разрешениях экранов. Тестирование клиентской части подтвердило корректность отображения данных, стабильность WebSocket-соединения и удобство пользовательского интерфейса.

В рамках курсового проекта была подготовлена техническая документация, включающая описание архитектуры системы, обоснование выбора технологий, диаграммы структуры и классов, а также руководство пользователя и администратора. Наличие данной документации обеспечивает прозрачность реализации и упрощает дальнейшее сопровождение и развитие программного продукта.

Разработанная система была успешно развернута и протестирована в тестовой среде, что подтвердило её работоспособность, надёжность и соответствие требованиям технического задания. Архитектура приложения позволяет в дальнейшем расширить функциональность системы, включая добавление постоянного хранилища данных, реализацию механизмов авторизации пользователей и масштабирование серверной части.

Таким образом, все поставленные цели и задачи курсового проекта были полностью выполнены, а созданное клиент-серверное приложение может быть использовано в качестве основы для систем мониторинга и визуализации телеметрических данных в реальном времени.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта было спроектировано и разработано клиент-серверное приложение для приёма, обработки и визуализации телеметрических данных в режиме реального времени. Полученный результат полностью соответствует требованиям технического задания и демонстрирует работоспособность выбранных архитектурных и технологических решений.

Разработанное приложение обладает простым и интуитивно понятным пользовательским интерфейсом, что позволяет использовать его без предварительной подготовки и обеспечивает наглядное восприятие телеметрических параметров. Автоматическое обновление данных и минималистичное оформление интерфейса повышают удобство работы с потоковой информацией.

Благодаря грамотно спроектированной клиент-серверной архитектуре система является масштабируемой и легко расширяемой. При необходимости в неё может быть добавлен дополнительный функционал, в том числе:

- сохранение телеметрических данных в постоянное хранилище (СУБД) для долговременного анализа;
- генерация отчётов и графиков по истории телеметрии за выбранные интервалы времени;
- поддержка нескольких источников телеметрии с одновременной обработкой данных;
- реализация системы авторизации и разграничения прав доступа пользователей;
- расширение набора обрабатываемых телеметрических параметров;
- интеграция с внешними системами мониторинга и аналитическими платформами;
- реализация мобильной версии или PWA-приложения для мониторинга телеметрии с планшетов и смартфонов.

Таким образом, разработанное клиент-серверное приложение может быть использовано в качестве основы для построения систем мониторинга и анализа

телеметрических данных, а также для дальнейшего развития и внедрения в более сложные программно-аппаратные комплексы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Таненбаум Э., Уэзеролл Д.
Компьютерные сети / пер. с англ. — 5-е изд. — СПб. : Питер, 2021. — 960 с. — ISBN 978-5-4461-0922-4.
2. Куроуз Дж., Росс К.
Компьютерные сети. Нисходящий подход / пер. с англ. — 8-е изд. — СПб. : Питер, 2022. — 912 с. — ISBN 978-5-4461-1866-0.
3. Басараб М. А., Колесников А. В.
Основы построения и моделирования компьютерных сетей : учебное пособие. — М. : МГТУ им. Н. Э. Баумана, 2020. — 120 с.
4. Сидоров А. В.
Сетевые протоколы TCP/IP : учебное пособие для вузов. — М. : Горячая линия – Телеком, 2019. — 304 с. — ISBN 978-5-9912-0674-3.
5. Гусев В. С.
Архитектура клиент-серверных приложений : учебное пособие. — СПб. : Лань, 2021. — 176 с. — ISBN 978-5-8114-4762-8.
6. Донован А., Керниган Б.
Язык программирования Go / пер. с англ. — М. : ДМК Пресс, 2021. — 432 с. — ISBN 978-5-97060-818-2.
7. Боднер М.
Go. Параллельное программирование и сетевые приложения / пер. с англ. — СПб. : Питер, 2022. — 384 с. — ISBN 978-5-4461-1934-6.
8. Мак-Карти М.
WebSocket: создание интерактивных веб-приложений реального времени / пер. с англ. — М. : ДМК Пресс, 2020. — 256 с. — ISBN 978-5-97060-725-3.
9. Филдс К., Вандербёрг С.
HTTP и WebSocket. Проектирование сетевых веб-приложений / пер. с англ. — СПб. : Питер, 2021. — 320 с. — ISBN 978-5-4461-1701-4.

10. Клеппман М.
Высоконагруженные приложения. Проектирование, масштабирование и сопровождение / пер. с англ. — СПб. : Питер, 2023. — 640 с. — ISBN 978-5-4461-2146-2.
11. Столлингс У.
Передача данных и компьютерные сети / пер. с англ. — 10-е изд. — М. : Вильямс, 2020. — 832 с. — ISBN 978-5-8459-1984-3.
12. Mozilla Developer Network.
WebSocket API [Электронный ресурс]. — URL: https://developer.mozilla.org/ru/docs/Web/API/WebSockets_API
13. The Go Authors.
Go Documentation: net, net/http, goroutines [Электронный ресурс]. — URL: https://go.dev/doc/
14. RFC 793.
Transmission Control Protocol (TCP). — Текст : электронный. — URL: https://datatracker.ietf.org/doc/html/rfc793
15. RFC 6455.
The WebSocket Protocol. — Текст : электронный. — URL: https://datatracker.ietf.org/doc/html/rfc6455